

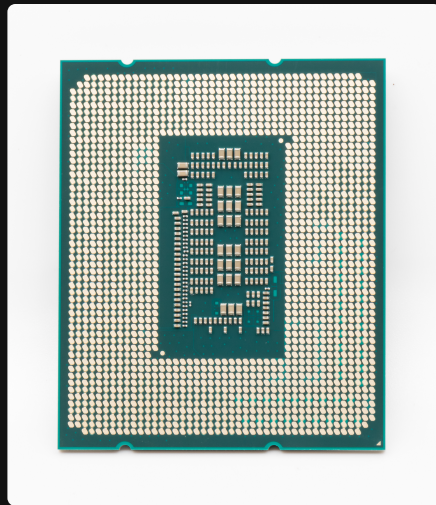
Processors

Overview

A computer consists of many moving parts, but the primary engine is the **Processor (CPU)**.

The CPU *executes* programs stored in main memory by continuously cycling through three steps:

1. **Fetching** instructions
2. **Decoding** (examining) them
3. **Executing** them sequentially one after another



The Essential Registers

While modern CPUs feature dozens of specialized registers, two remain absolutely vital to the Von Neumann architecture:

1. **Program Counter (PC)** Points to the memory address of the *next instruction* to be fetched.
2. **Instruction Register (IR)** Holds the *current instruction* while it is being decoded and executed.

The Instruction Execution Cycle

The continuous loop running under the hood of almost every computer is the **Fetch-Decode-Execute Cycle**:

1. **Fetch:** *Copy* the instruction from the memory address in the PC into the IR.
2. **Increment:** *Update* the PC to point to the next sequential instruction.
3. **Decode:** Determine the *type of instruction* and locate any required data in memory.
4. **Fetch Data:** *Pull* necessary data words into CPU registers (if required).
5. **Execute:** *Run* the instruction via the ALU/Control Unit.
6. **Repeat:** Loop back to Step 1.

Hardware vs. Software Equivalency

Because the Fetch-Decode-Execute cycle is *purely logical*, it doesn't *have* to be executed by physical transistors.

Any operation performed by software can be built directly into hardware, and any operation executed by hardware can be simulated by software.

If we write a software program to mimic this cycle, we call it an **Interpreter** or an **Emulator**.

This introduces a massive architectural choice for chip designers:

- do we build complex logic into physical wiring, or
- do we handle it via lower-level software?

The Shift to Microprogramming

In 1951, Maurice Wilkes introduced **Microprogramming** to bypass intense physical hardware limitations.

Instead of building massive, expensive circuits for complex instructions, designers created a "CPU within a CPU."

- *Macro-instructions* (complex user code) are intercepted.
- A tiny, built-in hardware interpreter breaks them down into simpler *micro-instructions* executed by the underlying hardware.

This shift led to the creation of the legendary **IBM System/360** (1964).

Benefits of Microprogramming

By moving complex instruction handling to an *internal microcode layer*, manufacturers realized several major advantages:

1. **Architecture Compatibility:** IBM introduced the term *architecture* because microprogramming allowed an entire family of machines (from cheap low-end models to expensive mainframes) to run the exact same software.
2. **Flexibility:** Allowed manufacturers to "hotfix" hardware bugs in the field via microcode updates.
3. **Cost-Effective Design:** New, complex instructions could be added at a minimal cost without redesigning the physical silicon.

By the late 1970s, almost all microprocessors used microcode to handle complex instructions.

RISC vs. CISC

By the late 1970s , processors had become **Complex Instruction Set Computers (CISC)**. They featured massive, highly complex instruction sets designed to bridge the "semantic gap" between machine code and high-level programming languages.

However, in 1980 , a group led by *David Patterson* and *Carlo Sequin* challenged this trend, introducing **Reduced Instruction Set Computers (RISC)** via the RISC I and RISC II architectures.

Feature	CISC (Complex)	RISC (Reduced)
Instruction Philosophy	Complex tasks in fewer, varied instructions.	Simple tasks executed fast using uniform instructions.
Execution	Uses microcode interpretation; variable clock cycles.	Directly executed by hardware; highly pipelined.
Memory Access	Many instructions can interact with memory.	Only explicit LOAD and STORE instructions touch memory.

RISC vs. CISC in Practice

look at how both architectures handle multiplying numbers stored in memory:

- CISC Approach (Single, complex instruction)

```
1  MULTIPLY [Memory Location A], [Memory Location B]
```

- RISC Approach (Multiple, simple instructions)

```
1  LOAD  Register 1, [Memory Location A]
2  LOAD  Register 2, [Memory Location B]
3  PROD  Register 1, Register 2
4  STORE Register 1, [Memory Location A]
```

Even though RISC requires *more lines of code*, its uniform, simple instructions execute so **quickly** (often 1 cycle per instruction) that it **outpaced** pure CISC performance.

The Modern Landscape

While desktop computing was *historically dominated* by x86 (CISC) due to backward compatibility and legacy software investments, the modern landscape is heavily defined by a **synthesis** of both:

1. **Hybrid Cores:** Modern x86 processors (Intel/AMD) are actually hybrids.
 - They feature a CISC outer shell that decodes legacy instructions into internal, RISC-like "micro-ops."
2. **RISC Dominance:** RISC *is* mainstream just not in consumer computers.
 - *ARM* (a RISC architecture) powers virtually the entire mobile ecosystem,
 - modern supercomputers,
 - major cloud data centers, and
 - Apple Silicon laptops/desktops.
3. **Open-Source Hardware:**
 - **RISC-V** is rapidly expanding across the industry as an open-standard RISC ISA.

Principles of RISC

RISC *is the future*, and so the design decisions of RISC also because the new way to architect general-purposes CPUs

1. **Direct Hardware Execution:** Maximize instructions executed directly by hardware to avoid microcode overhead.
2. **Maximize Issue Rate:** Optimize pipelining to start executing new instructions before previous ones finish.
3. **Easy Decoding:** Keep instructions at a fixed length with uniform formats to make parsing instant.
4. **Isolated Memory Access:** Only explicit `LOAD` and `STORE` instructions should touch RAM; all calculations must happen inside registers.
5. **Abundant Registers:** Provide plenty of registers to minimize the need to wait on slow system memory.